CORNDEL

AI6 WORKSHOP 6 – OPTIMISATION (MATHS MADE PRACTICAL)

Making Things Better with Model Tuning

Before we start

Safeguarding: if you have concerns about a colleague or learner during today's session, please speak to your coach straightaway; they are here to help and will know how to take it forward. If you are unsure about your organisation's reporting process, ask your coach now.

Wellbeing check-in: take 30 seconds and turn to the person next to you: how are you doing today, *honestly*?

And one for yourself: what is one kind thing you have done for yourself this month?

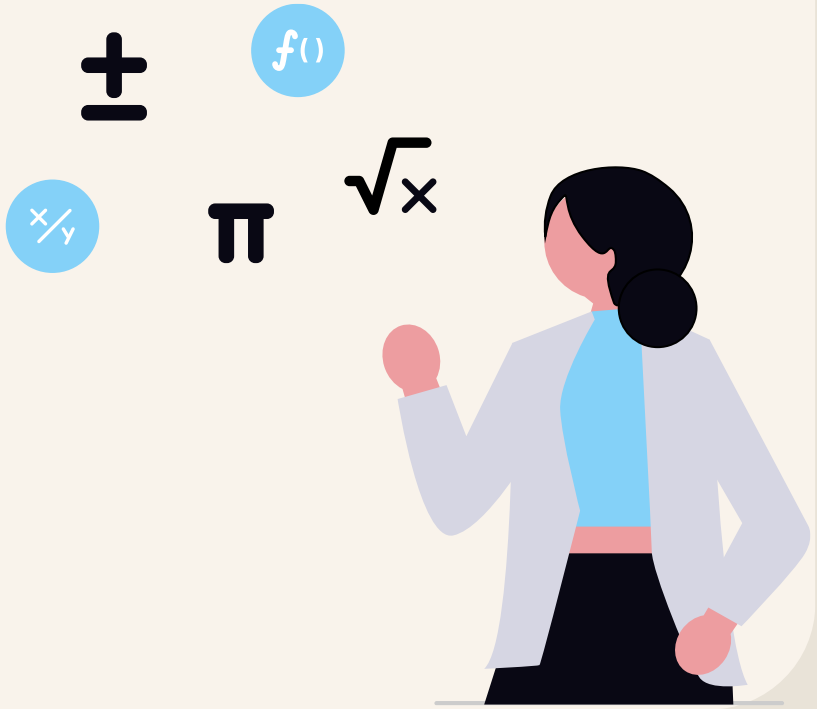
There is no right answer. You do not have to share with the room. We ask because this work takes energy, and noticing how you are arriving matters.

What you will learn today

- **What fine-tuning is (and isn't):** optimisation from a strong starting point
- What happens inside **trainer.train():** *loss → gradients → weight updates.*
- How to diagnose learning rate issues from curves and metrics.
- How to spot and explain the generalisation gap (train vs **validation**).

Unit 6 “Maths” without the pain

- **We won’t derive calculus on a whiteboard…**
- …but we will use the real words: loss, gradient (partial derivative), SGD, learning rate, generalisation
- Why: tools and papers use this vocabulary, and engineers need to speak it



Workshop Structure — Activities at a Glance

Activity 1	Warm-Up: Curve Diagnosis	— Catch up; read curves; discuss what’s familiar (B3, K18)
Activity 2	Environment Setup and Baseline	— Record your 'before' measurement
Activity 3	Group Learning Rate Experiment	— Run your preset, log the results
Activity 4	Share and Compare	— Whole-group synthesis and engineering conclusion
Activity 5	Convergence Check	— Maths to compute cost (K18)
Activity 6	Data Quality and Uncertainty	— Stakeholder caveats (B5)
Activity 7	Reflection and Horizon Scan	— Task 6 materials (K18, S31, B1)
Activity 8	Going Further (optional)	— Extensions: schedule, freeze, error analysis

Fine-tuning vs retraining vs optimisation

Optimisation
(the "mechanism")

- Reduce a loss function by adjusting weights
- Uses gradients (slopes / partial derivatives)
- Runs in tiny steps, thousands of times

Fine-tuning vs retraining
(the context)

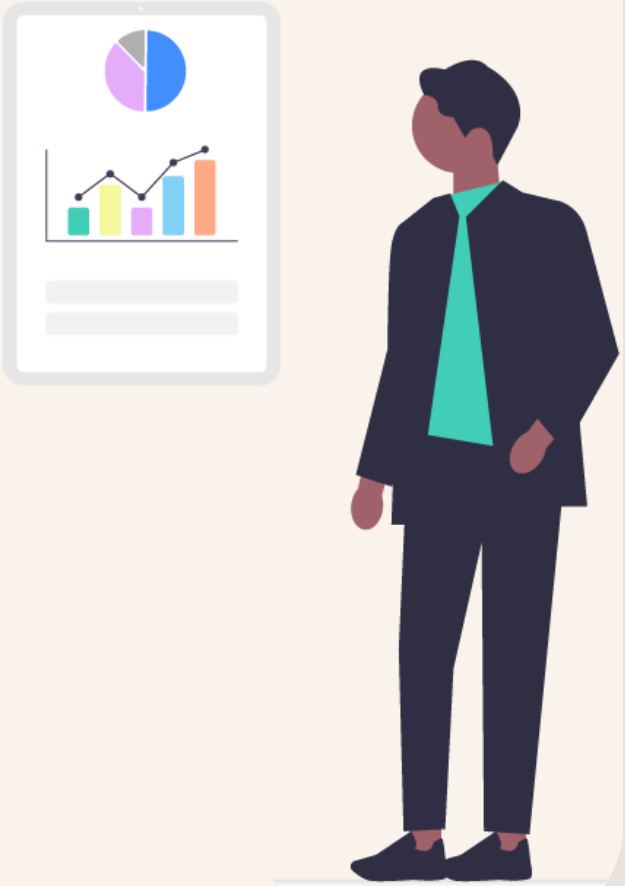
- Fine-tuning: start from a pretrained model
- Retraining: run optimisation again because data changed
- Training from scratch: rare + expensive in business

What trainer.train() does

- Reads a mini-batch of labelled examples
- Predicts labels
- Scores wrongness (loss)
- Computes gradients (how to reduce loss)
- Updates weights using an optimiser (e.g. AdamW — in the SGD family)
- **...and the loop runs thousands of times until loss stops improving**

Metrics: what to watch

- Loss = altitude (are we going downhill mathematically?)
- Validation accuracy/F1 = road test (does it work on unseen examples?)
- Generalisation gap
  = train accuracy - validation accuracy
- Gap widening = likely overfitting (memorising)



You've seen this before!

Iterative adjustment against a target is not unique to machine learning!

Too cautious (too low): *a new team member given only tiny tasks — they technically improve, but so slowly it barely registers. A medication dose raised by a fraction each month when a bolder step is needed.*

Too aggressive (too high): *an interest rate raised 5% in one step — overshoots, causes recession. A diet changed by cutting 1,000 calories overnight — unsustainable, collapses.*

Just right: a physio programme that stretches but does not injure — steady measurable gains, consistent across sessions.

Generalisation gap: a revision technique that works perfectly on past papers but fails in the actual exam. The student memorised the questions, not the subject.

Today you will watch these same patterns happen live inside a training loop — with numbers you can measure!

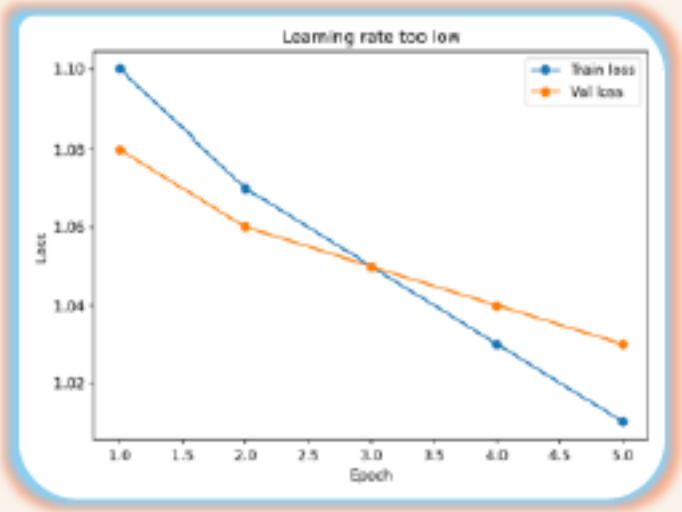
Activity 1 — Warm-Up: Curve Diagnosis

this is a warm-up, not a lab

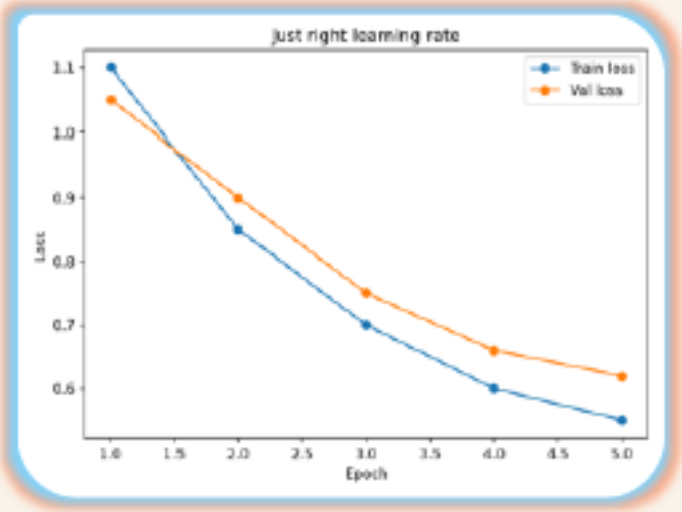
Task 1: catch up — what have you been working on since last workshop?

Task 2: match each curve to its cause — TOO_LOW / JUST_RIGHT / TOO_HIGH *(see next slide)*

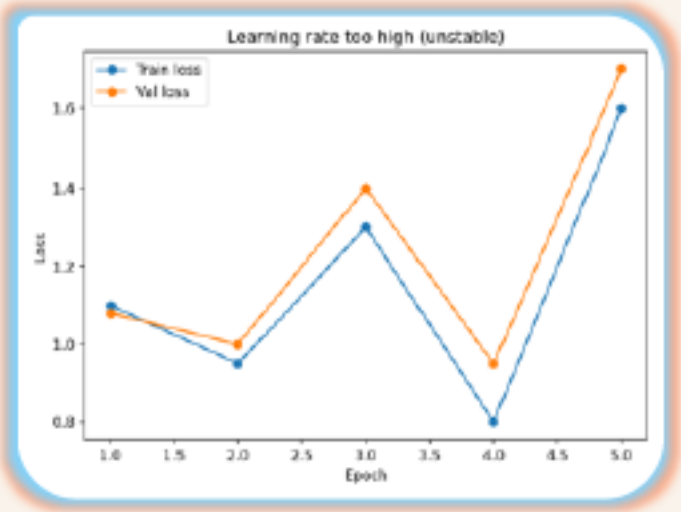
Reading curves (diagnostic)



Too Low LR
(slow learning)

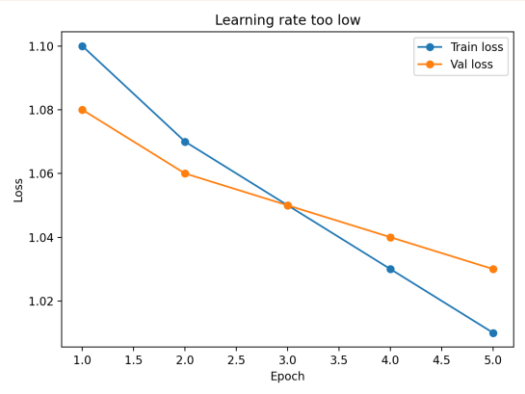


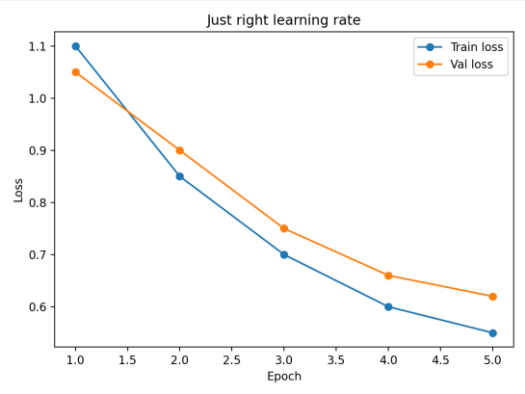
Just right
(steady
improvement)

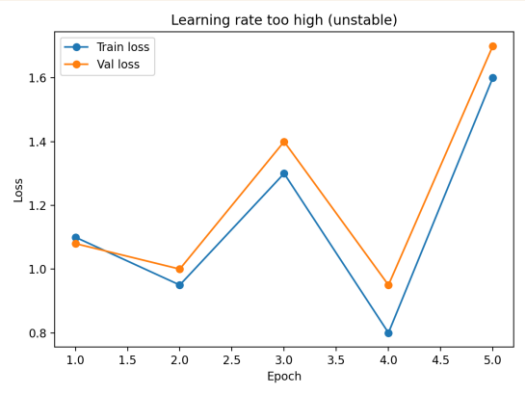


Too high LR
(unstable)

Activity 1 — Warm-Up: The Three Patterns (**Task 2**)







Too low LR
(slow learning)

Just right
(steady improvement)

Too high LR
(unstable)

Task 3: which of the examples from the previous slide feels familiar from your work? Share briefly

Task 4 (B3): explain one of these patterns to someone non-technical, no jargon.

KSBs: K18 (reading signals), B3 (communicating technical and non-technical backgrounds)

Activity 2 — Environment Setup and Baseline

- Clone repo, create virtual environment, install requirements (35 minutes)
- Launch Jupyter Lab and open the main notebook
- Run baseline evaluation cells — record metrics BEFORE any training
- Locate and understand the speed dials (TRAIN_SUBSET, MAX_LENGTH, EPOCHS)

Checkpoint: baseline accuracy and F1-macro recorded in your experiment log

KSB: S9 (before-and-after comparison — this is your 'before')

Activity 3 — Group Learning Rate Experiment

Groups of 3–4, each running ONE preset: *TOO_LOW, JUST_RIGHT*, or *TOO_HIGH*

Before you run: write your prediction. Assign roles — driver, logger, observer.

0–25 min: run training, record metrics each epoch as they appear

25 min — midpoint pulse: each group calls one word across the room: “moving”, “stalled”, or “crashed”

25–50 min: inspect curve, complete experiment log, run extension if time allows

Your coach(es) will be circulating throughout — flag them if you are stuck or if training is very slow

KSBs: K18 (loss and gradients in action) | S9 (logging for reproducibility)

Activity 3 — Group Learning Rate Experiment

What you run

Each group picks one preset:
TOO_LOW, JUST_RIGHT, or TOO_HIGH
Run 1–2 epochs and record metrics

What you expect

TOO_LOW: curve barely moves
JUST_RIGHT: steady improvement
TOO_HIGH: unstable / oscillation / may error

Activity 4 — Share and Compare

Each group reports results into the shared slide (30 minutes)

Compare curves across TOO_LOW / JUST_RIGHT / TOO_HIGH

which preset would you recommend for production… and why?

Task: share any patterns from today that felt familiar from your own work; the whole group benefits from hearing the experience from different industries!

Stakeholder framing (B3): draft a one-paragraph update for a non-technical colleague: translate the result, do not just report the numbers

KSBs: K18 (generalisation gap) | B3 (communicating across technical and non-technical backgrounds)

Activity 4 — Group Results *(Sharing slide to use)*

Copy your group's numbers from your experiment log into the row for your preset.

Preset	LR value	Train loss (end)	Val loss (end)	Accuracy	F1-macro	Gen. gap (accuracy)	Notes
TOO_LOW							
JUST_RIGHT							
TOO_HIGH							

1. Compare the curves — did predictions match results?
2. Which preset had the healthiest generalisation gap?
3. If choosing one for production: which, and why?

Gen. gap = train accuracy − val accuracy | Use accuracy or F1-macro, not loss, so signs are consistent across groups

Activity 5 — Convergence Check

Convergence = the point where further training is no longer producing meaningful improvement.

Like a kettle approaching boiling point: the temperature is still rising, but each second adds less than the last. At some point, waiting longer gives you nothing more.

We measure it with loss delta: how much did loss change between the last two epochs? If the answer is very small (e.g. < 0.01), the model has converged within our threshold.

Why it matters: reliability (a model still moving is not stable for deployment) + cost (training past convergence burns compute for no gain).

Run the convergence check cells (6b), record your delta, and answer: would more epochs be worth the cost for this task?

KSB: K18 — connecting mathematical principles (loss, gradients) to a practical engineering decision (when to stop)

Activity 6 — Data Quality and Uncertainty

Whole-group facilitated discussion

financial_phrasebank (*allagree* subset):
only the unambiguous cases — not representative of real text

synth_financial_sentiment.csv: synthetic — generated, not collected

Small subsets (300-800 examples): 2% accuracy difference may be noise

Deployment question: what caveats would you give a project manager?

KSBs: B5 (data quality and uncertainty), S14 (statistical analysis — margins)

Activity 7 — Reflection and Horizon Scan

write reflections individually

Use the vocabulary from today: loss, gradient, generalisation gap, convergence

find ONE credible horizon scan source relevant to today

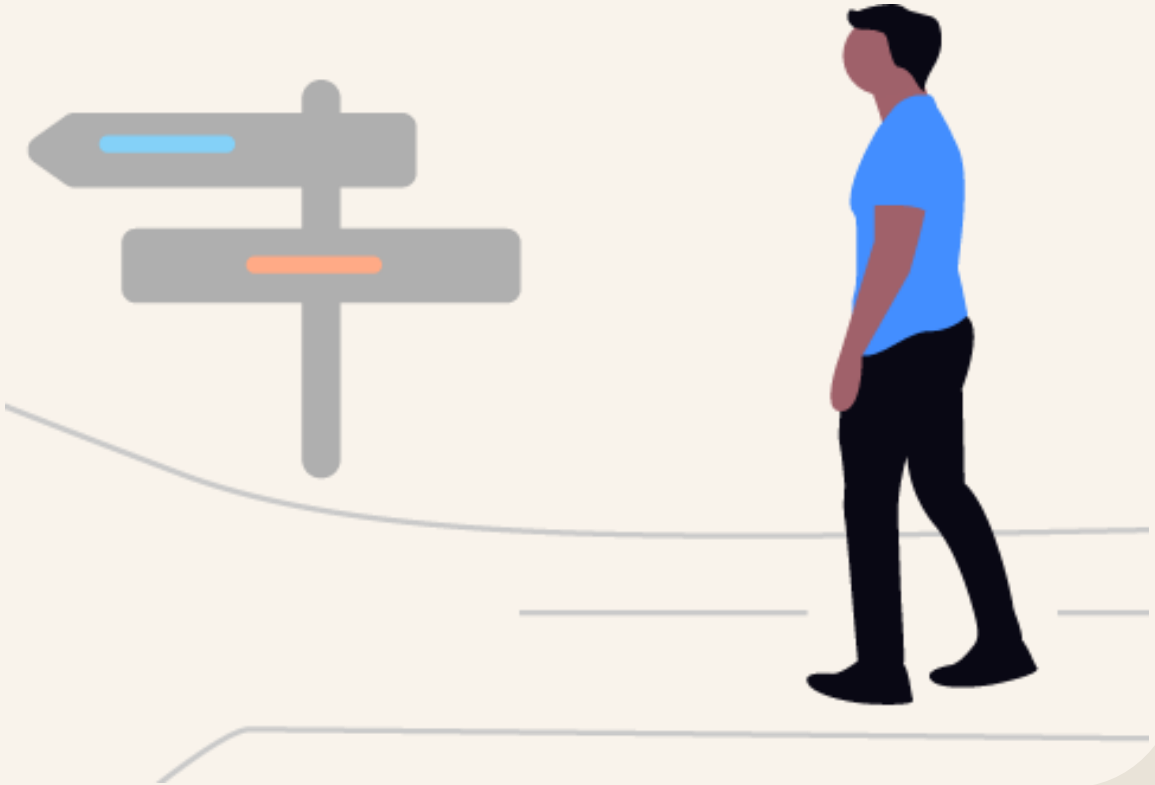
Suggested searches: LoRA fine-tuning, learning rate schedules, training carbon footprint

Volunteers may want to share what they found and why it is useful

KSBs: K18, B1 (self-directed learning), S31/K28 (horizon scanning for Task 6)

Going further (optional)

- Schedule: constant vs linear learning rate schedule
- Freeze encoder: train only the classification head
- Error analysis: confusion matrix + inspect misclassifications
- Experiment tracking (optional): W&B if policy allows



Practical tuning rules (so you don’t waste time)

- Always take a baseline
- Change ONE dial at a time (start with learning rate)
- Time-box experiments (compute is money)
- Write a short decision log (what you tried, what happened, what you’ll do next)

Wrap up

- You can now explain fine-tuning using the optimisation loop
- You can diagnose learning rate issues from curves
- You can describe generalisation gap and why it matters
- Next: connect this to Unit 6 evidence (Task 6 / CPD q’s)

We’d love your feedback:

https://qr2.mobi/273dl